

Feature-by-Feature Guide & Test Oracles

Product: **Que** (SchemaGraph) — join truth → human approve → freeze → attested PR

Codebase: `adc/schemagraph` · SPA default `http://localhost:5173` · API `http://localhost:8787`

Document version: **2.0** · Date: 29 Jul 2026

Audience: QA, design partners, engineers · Companion checklist: `docs/MANUAL_TEST_PLAN.md`

This document explains **every major feature**: what it is for, how it must work, how to test it, and what a **correct response** looks like (UI + API). Use it as the source of truth for functional / production-level validation before a paid POC.

Wedge (sell this): Cross-source join discovery with human-in-the-loop review, contract freeze, and HMAC-attested dbt PRs — schema metadata only (no warehouse row lake).

Avoid claiming for now: SOC2 Type II, SCIM, full catalog replacement this quarter.

Table of contents

1. How to use this guide
2. F1 — Password auth & sessions
3. F2 — OIDC SSO (PKCE, JWKS, hash token)
4. F3 — Workspaces, roles & invites
5. F4 — Sources & credential encryption
6. F5 — Sync, canvas & layout
7. F6 — Join inference, promote / reject, stitch
8. F7 — Privacy & sample scrub
9. F8 — Jobs notebook, run, validate, export
10. F9 — Contracts, drift & export gates
11. F10 — Attestation (HMAC)
12. F11 — Workspace settings & BYOK secrets
13. F12 — AI chat & RAG
14. F13 — BI lineage & dbt manifest assist
15. F14 — Health, CORS, OpenAPI, production boot
16. End-to-end happy path (design partner)
17. Role permission matrix
18. Environment reference
19. Automation commands

0. How to use this guide

For each feature section you get four blocks:

Block	Meaning
PURPOSE	Why the feature exists in the product wedge
HOW IT WORKS	Correct runtime behavior (inputs → processing → outputs)
CORRECT RESPONSE	Oracle: UI state and/or HTTP status + JSON fields that must appear
HOW TO TEST	Manual steps + common fail oracles

Mark results in `docs/MANUAL_TEST_PLAN.md`. Run automation first: `cd api && npm run test:diligence`.

F1 — Password authentication & sessions

Purpose

Gate every workspace behind identity. Sessions are Bearer tokens; roles enforce write/admin actions.

How it works

1. User opens `/login` — email/password fields start **blank**.
2. Demo account shortcuts appear only in **Vite DEV** builds (not production UI).
3. `POST /auth/login` verifies script password hash → creates session (token hashed SHA-256 in DB).
4. SPA stores token and sends `Authorization: Bearer <token>` on API calls.
5. In production, `STITCH_AUTH_DISABLED` is ignored by `authDisabled()` and boot refuses if set.

Correct response — login success

HTTP 200:

```
{
  "ok": true,
  "token": "<64+ hex>",
  "expiresAt": "ISO-8601",
  "user": { "id": "uuid", "email": "...", "displayName": "... " },
  "workspaces": [{ "id": "uuid", "name": "...", "slug": "...", "role":
    "owner|admin|member|viewer" }]
}
```

UI: redirect to `/workspace` (or prior `from` route). No demo passwords visible in prod build.

Failure oracles

Wrong password → **401** `invalid email or password`, stay on login.

Expired/missing Bearer → **401** `unauthorized`, SPA returns to login.

Prod boot with `STITCH_AUTH_DISABLED=true` → process exits before listen.

How to test

1. Open login — confirm blank fields; prod build has no “DEMO ACCOUNTS” block.
2. Valid owner credentials → lands in workspace; TopBar shows user.
3. Logout → protected routes redirect to `/login`.
4. Optional: set `QUE_ENV=production` + `STITCH_AUTH_DISABLED=true` — API must refuse to start.

F2 — OIDC SSO (PKCE · JWKS · hash token · provisioning)

Purpose

Enterprise sign-in without putting session tokens in query strings or auto-joining a demo workspace.

How it works

1. `GET /auth/sso` reports readiness. If ready, Login shows "Continue with SSO".
2. `GET /auth/sso/start` stores PKCE `code_verifier` in Postgres table `oidc_pending_states`, redirects to IdP.
3. Callback exchanges code, verifies `id_token` via JWKS (RS256), provisions/updates user.
4. Membership: accept pending invites; enforce `QUE_SSO_ALLOWED_DOMAINS`; if `QUE_SSO_REQUIRE_INVITE=true`, new users need an invite; optional `QUE_SSO_DEFAULT_WORKSPACE_ID` only when configured — **never silent demo auto-join**.
5. Redirect to SPA: `... /auth/callback#token= ... &expiresAt= ...` (hash fragment).
6. SPA reads hash, calls `acceptToken`, then `history.replaceState` strips the URL.

Correct response

`GET /auth/sso` → `sso.loginImplemented: true, status: "ready"`.

Browser address after IdP: hash contains `token=`; **query must not** contain `token`.

After callback page settles: URL is `/auth/callback` or `/workspace` with **no token** left.

User has membership in invited / allowlisted workspace only.

Failure oracles

`?token=` in query → SPA error: token must not be in query string.

Domain not allowlisted → **403** SSO email domain not allowed.

Invite required, none present → **403** No workspace invite...

Expired state → **400** invalid or expired OIDC state.

How to test

1. Configure OIDC env (see §17); restart API; confirm SSO button.
2. Complete login; watch Network/Address bar for `#token=` only.
3. Create invite via API as admin; SSO with new email → joins correct role.
4. SSO with disallowed domain → blocked.

F3 — Workspaces, roles & invites

Purpose

Multi-tenant isolation. Every resource is scoped by `workspaceId` + membership role.

How it works

- Roles (ascending): `viewer` < `member` < `admin` < `owner`.
- Writes (sync, jobs, promote) require $\geq$ `member`; connection create / settings / invites $\geq$ `admin`.
- Invites API: `GET|POST|DELETE /workspaces/:id/invites` (admin). Claimed on next login/SSO.
- Settings “Members” UI shows the **current user** (real ACL), not a fake roster.

Correct response — create invite

```
HTTP 201: { "ok": true, "invite": { "id", "email", "role", "createdAt", "acceptedAt": null } }
```

How to test

1. As viewer: attempt create connection → **403**.
2. As admin: `POST ... /invites` with email + role → 201; list shows invite.
3. Login/SSO as that email → invite disappears (`acceptedAt` set) and workspace appears in `/auth/me`.
4. Access another workspace UUID → **403** not a member.

F4 — Sources & credential encryption

Purpose

Connect warehouses (Postgres, Databricks, Snowflake) and files without storing plaintext secrets in `config_json`.

How it works

1. Admin creates connection on `/sources` (type + host/catalog/token/password).
2. API seals secret fields into AES-GCM `__enc` blob (`QUE_SECRETS_KEY`).
3. Public GET returns redacted config (`.....`) + `hasSecrets: true`.
4. Sync / live exec unseals in memory only; blank password on PATCH keeps previous secret.
5. Modes: `live` or `fixture` (demo JSON for DBX/SF without real warehouse).

Correct response — create

HTTP **201**: connection object with `syncable: true` for PG/DBX/SF, `hasSecrets: true` when token/password provided.

DB check: `connections.config_json` contains `__enc`; must **not** contain raw `token` / `password` strings.

Duplicate name → **409**. Bad live credentials on sync → connection `status: error`, sync **502** introspect failed.

How to test

1. Create Snowflake/Databricks fixture connection → Sync → tables appear on canvas.
2. Inspect DB row for `__enc` only.
3. Edit connection leaving password blank → sync still works.
4. Delete connection → removed from Sources and schema.

F5 — Schema sync, canvas & layout

Purpose

Show tables as a graph: sources on the left, draggable nodes, edges for relationships.

How it works

1. Sync introspects columns (+ optional capped samples) into `schema_objects` / `schema_columns`.
2. `GET /workspaces/:id/schema` returns tables + non-rejected relationships + saved positions.
3. Drag nodes → `PUT /layout` merges positions into `diagram_layouts`.
4. Right sidebar shows selected table columns / samples (scrubbed if policy on).

Correct response — schema

HTTP **200**: `tables[]` with `id`, `name`, `sourceId`, `sourceType`, `columns[]`, `position`; `relationships[]` with `status` in `suggested|accepted|explicit` (rejected omitted).

After reload, node positions match last drag.

How to test

1. Sync two sources → both appear in left sidebar filters.
2. Drag a table, hard-refresh → position restored.
3. Select table → right panel columns match source names/types.

F6 — Join inference, promote / reject, stitch session

Purpose

Core wedge: discover cross-source joins as *suggestions*; humans promote before freeze/export.

How it works

- On sync (if `inferJoinsOnSync`): multi-signal name/type scoring creates `status=suggested` edges — **never auto-accepted**.
- Databricks live: optional query-history SQL JOIN parse (`databricksQueryJoinAssist`).
- Promote → `accepted`, confidence 1, audit event; Reject → hidden from graph/export.
- Stitch session: pick two connections → infer → optional job + dbt PR (blocked until ≥ 1 promoted join between them).

Correct response — promote

HTTP 200: { "ok": true, "relationship": { "id", "status": "accepted", "confidence": 1, ... } }

Edge style on canvas updates from suggested → accepted.

Join inference: { "ok": true, "created": N, "scanned": M, "durationMs": ... }

Stitch ship without promoted join → **409** Ship blocked: promote at least one join...

How to test

1. Sync two fixtures → suggested edges appear.
2. Promote one → stays after reload; Reject another → gone from schema GET.
3. Stitch session dialog: run inference; try ship before promote → 409; after promote → job/export proceeds.

F7 — Privacy & sample scrub

Purpose

Schema-only posture: if samples are collected, tokenize before metadata DB storage.

How it works

- Defaults: `includeSamplesDefault: false`, `scrubSamples: true`.
- When samples on + scrub on: emails → `email_...`, UUIDs → `uuid_...`, free text → `tok_...`.
- Live validate / chat sample previews are capped (≤ 20 / ≤ 5) and must not dump warehouse rows into prompts.

Correct response

After sync with scrub: column sample arrays in DB/UI look tokenized, not raw PII.

Settings toggles Save → GET settings echoes same booleans.

How to test

1. Settings → enable include samples + scrub → Save → sync → inspect samples.
2. Disable scrub → re-sync (dev only) → rawer values (still capped).
3. Automation: `npm run test:privacy` must PASS.

F8 — Jobs notebook (run · validate · ready · export)

Purpose

Turn reviewed joins into a notebook job, freeze a schema-only contract, and export attested artifacts / dbt PRs.

How it works

1. Create job from canvas/stitch or blank → freezes contract (tables + accepted joins + column types + snapshot id).
2. Edit markdown + SQL cells (light notebook UI on `/jobs`).
3. **Run Test** (`mode: dry_run`): lint + schema sample previews — no warehouse write.
4. **Validate** (`mode: validate`): read-only live SQL, hard cap 20 rows; blocks DML/DDL/multi-statement.
5. Mark **ready** → Export `json | sql | dbt | dbt-pr`.

Correct response — run

HTTP **201**: `run.status` success/failed; `output.cellResults` / `liveResults` present; logs array non-empty for failures.

Export JSON HTTP **200**: includes `export.attestation` with `policy: "schema-only"` and `signature.alg: "HMAC-SHA256"` (prod keys set).

`DELETE FROM ...` / multi-statement → blocked by `prepareReadonlySql`.

Export with open high drift / unreviewed joins / column drift (flags on) → **409** + `validation.blocking: true`.

How to test

1. Create job with SQL cell → Run Test → Process/Output populate.
2. Validate against live PG if available → ≤20 rows.
3. Export JSON → copy attestation → `POST /auth/attestation/verify` → `{ "ok": true }`.
4. With drift open + block flags → dbt-pr returns 409; ack / clear → PR opens (GitHub configured).

F9 — Contracts, drift & export gates

Purpose

Frozen contract is the truth for export. Drift detection blocks shipping until humans acknowledge or fix.

How it works

- Sync computes drift events (`info|warn|high`) vs previous snapshot.
- Settings: `blockExportOnDrift`, `blockExportOnUnreviewedJoins`, `blockPrOnColumnDrift`.
- `GET ... /contract/validate` previews gates without exporting.
- `force: true` on export bypasses gates but still writes audit (footgun — admin only awareness).

Correct response — drift list

```
{ "ok": true, "events": [ ... ], "openHigh": [ ... ], "hasBlockingRisk": boolean }
Ack: 200 { "ok": true, "id": " ... " } — event no longer blocking.
```

How to test

1. Toggle block flags in Settings → Save.
2. Introduce suggested join on job tables → export 409.
3. Promote/reject → export succeeds.
4. Validate contract endpoint mirrors export gate messages.

F10 — Schema-only attestation (HMAC)

Purpose

Prove an export was produced under schema-only policy with listed approved joins — verifiable offline via HMAC.

How it works

- Every export embeds attestation v2: policy, snapshot id, approved relationship ids, guarantees, claim, fingerprint.
- Signed with `QUE_ATTESTATION_HMAC_SECRET`.
- `POST /auth/attestation/verify` is public — recompute hash + HMAC.

Correct response — verify

Body: paste `export.attestation` object.

HTTP 200: { "ok": true, ... }.

Tamper any field → { "ok": false, "reason": "payload hash mismatch" | "bad signature" }.

How to test

1. Export job → verify attestation → ok true.
2. Flip one character in claim → verify fails.
3. Automation covers sign/verify in `test:functional` / `test:privacy`.

F11 — Workspace settings & BYOK secrets

Purpose

Per-workspace policy surface for privacy, drift blocks, Databricks assist, GitHub dbt path, and encrypted LLM/GitHub keys.

How it works

- `GET/PATCH /settings` — admin for writes.
- Policy toggles include scrub, infer joins, DBX query-history, block export/PR on drift, unreviewed joins.
- `PUT /secrets/llm` stores OpenAI/Anthropic/GitHub token encrypted; status returns `source: workspace|env|none` only (never raw key).

Correct response — settings GET

Includes `settings`, `stats`, `capabilities` (connectors, llm, secrets, sso, github, wedge note).

After Save: dirty flag clears; Discard restores draft including scrub / blockPr / databricks flags.

How to test

1. Toggle scrub + block dbt PR on column drift + DBX assist → Save → reload → values persist.
2. Save GitHub token → secrets status shows workspace source.
3. Viewer cannot PATCH settings (403).

F12 — AI chat & RAG

Purpose

Answer schema questions and draft jobs from metadata packs — not from centralized row data.

How it works

- `POST ... /chat` builds schema context; optional pgvector RAG; skills (`/list`, `/joins`, ...) or LLM if `preferLlmChat` + keys.
- `@mentions` insert table/column context in the composer.
- Feedback `rating: 1|-1` stored for RLHF-lite.

Correct response

```
{ "ok": true, "reply": "...", "citations": [ ... ], "mode": "skill|llm|...", "vectorReady": boolean }
```

Without LLM keys: still answers via skills/heuristics (not a hard failure).

How to test

1. Ask "list tables" / use `/joins` — reply cites workspace tables.
2. `@mention` a table — token inserted; reply references it.
3. Confirm reply does not paste large raw sample dumps.

F13 — BI lineage & dbt manifest assist (API)

Purpose

Thin reverse-lineage ingest and dbt manifest matching — assist joins/context; not a full BI catalog product.

How it works

- `POST ... /bi-lineage` with tool + assets → match tables → snapshot.
- `POST ... /dbt-manifest` with manifest JSON → extract refs → matched tables.

```
BI: { "ok": true, "linked": N, "preview": [ ... ] } (N may be 0 if no match — still ok).
```

```
dbt: { "ok": true, "edges": [ ... ], "matchedTables": [ ... ] }
```

How to test — use curl/Postman with Bearer token; then `GET ... /bi-lineage` lists recent items.

F14 — Health, CORS, OpenAPI & production boot

Purpose

Ops readiness and browser lock-down for paid POC hosting.

How it works

- `GET /health` pings DB; reports `authDisabled` and SSO status.
- CORS allowlist from `QUE_CORS_ORIGINS` (required in production).
- Prod boot requires `QUE_SECRETS_KEY`, `QUE_ATTESTATION_HMAC_SECRET`, `QUE_CORS_ORIGINS`; refuses auth bypass.
- TLS: terminate TLS 1.2+ at reverse proxy (documented; not in Node itself).

```
Health 200: { "ok": true, "service": "que-api", "authDisabled": false, "sso":  
"ready" | "not_configured" }
```

Allowed origin: browser XHR succeeds. Disallowed origin: CORS blocked.

How to test

1. `curl http://localhost:8787/health` and `/openapi.json`.
2. Set CORS to your SPA origin only; open from other origin → blocked.
3. Prod-like env missing key → process exits with clear error.

15. End-to-end happy path (design partner)

1. **Provision** — migrate DB (incl. `014`), set secrets, start API + SPA.
2. **Auth** — login or SSO with invite; confirm role.
3. **Sources** — add SF + DBX (fixture or live); confirm `__enc`; Sync both.
4. **Joins** — review suggested edges; Promote gold joins; Reject noise.
5. **Privacy** — scrub on; samples tokenized if collected.
6. **Job** — create notebook from stitch; Run Test; Validate; Ready.
7. **Gate** — contract validate clean; export JSON; verify attestation.
8. **Ship** — Open dbt PR (GitHub token + owner/repo set); PR body references attestation.
9. **Regression** — change column → drift high → export 409 until ack/fix.

Pass criteria for paid POC: steps 1–8 complete with evidence; no plaintext warehouse secrets; no token in query string; attestation verifies.

16. Role permission matrix

Action	viewer	member	admin	owner
View schema / jobs / chat read	✓	✓	✓	✓
Sync / promote-reject / run job / export	—	✓	✓	✓
Create/edit connections	—	—	✓	✓
Patch settings / secrets / invites	—	—	✓	✓

Exact gates: `requireMinRole` in `api/src/auth.js` + route wiring in `api/src/index.js`.

17. Environment reference (paid POC)

Variable	Required in prod?	Notes
<code>QUE_SECRETS_KEY</code>	Yes	AES key for connections + workspace secrets; ≠ default string
<code>QUE_ATTESTATION_HMAC_SECRET</code>	Yes	Export HMAC
<code>QUE_CORS_ORIGINS</code>	Yes	Comma-separated SPA origins
<code>QUE_ENV=production</code>	Yes (for gate)	Enables boot assertions
<code>QUE_SEED_DEMO_USERS</code>	Set false	Do not seed demo passwords in POC
<code>STITCH_AUTH_DISABLED</code>	Must be unset/false	Boot refuses if true in prod
<code>QUE_OIDC_*</code> / <code>QUE_SSO_*</code>	If SSO	See F2
<code>VITE_STITCH_API_URL</code>	SPA	API base URL

Full template: `api/.env.production.example`.

18. Automation commands

```
cd adc/schemagraph/api
npm run test:diligence      # joins golden-set + privacy + functional/security
npm run test:smoke          # optional live API smoke (needs running server)
cd ..
npx tsc --noEmit -p tsconfig.app.json

# Manual PASS/FAIL sheet
docs/MANUAL_TEST_PLAN.md

# CI
adc/.github/workflows/que-diligence.yml
```

Que Feature Guide v2.0 · Generated for paid-POC readiness · Related: [Que-Technical-Documentation.html](#), [Que-Schema-Only-Attestation.html](#), [MANUAL_TEST_PLAN.md](#).

Print this file to PDF (browser Print → Save as PDF) or use the companion [Que-Feature-Guide.pdf](#) in the same folder when generated.